An **AI Flashcard and Smart Notes Maker** is a good project to demonstrate your AI/ML capabilities. In fact, it could become a strong portfolio project if you build it as more than a basic PDF-to-summary application.

My suggestion is to make it an **AI-powered learning platform that combines multiple ML models, local LLMs, and intelligent content generation**.

The key is to show that you understand how to select, integrate, evaluate, and optimise AI models — not just call an LLM API.

Project concept: NeuroLearn AI

Strong AI/ML portfolio potential

One-line description:

An intelligent study assistant that converts PDFs, lecture slides, textbooks, and notes into structured short notes, flashcards, quizzes, and personalised revision plans using multiple ML models and locally running LLMs.

Core features



Smart short-notes generation

Upload PDFs, PPTs, DOCX files or images. Extract content, identify important concepts, and generate structured notes with headings, definitions, examples and key points.



AI flashcard generation

Generate question-answer cards, definition cards, concept cards and application-based cards. Allow users to edit and save them.



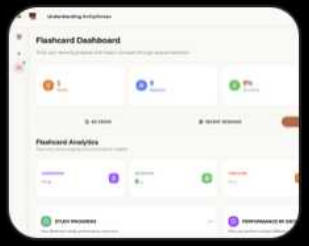
Adaptive quiz generation

Create MCQs, fill-in-the-blanks and short-answer questions. Adjust difficulty based on previous answers.



Concept and topic mapping

Identify relationships between concepts and create a topic-wise knowledge graph.



Personalised revision

Track correct and incorrect answers, estimate topic mastery, and schedule revision using spaced repetition.

How to demonstrate different AI/ML models

This is the most important part of your project. Instead of using one LLM for everything, build a pipeline where each model has a specific responsibility.

AI component	Model / technology	Purpose
Text extraction	PyMuPDF, Tesseract	Extract text from documents and scanned pages
Text classification	DistilBERT	Classify content into topics
Keyword extraction	KeyBERT	Identify important concepts
Summarisation	FLAN-T5 / local LLM	Generate short notes
Embeddings	Sentence Transformers	Understand semantic similarity
Question generation	T5 / local LLM	Generate questions and flashcards
Reranking	Cross-encoder	Select relevant and high-quality cards
Topic clustering	K-Means / HDBSCAN	Group related concepts
Personalisation	Spaced repetition algorithm	Schedule revision

These are candidate models and technologies to experiment with, not a requirement to use all of them.

Local LLM integration

You can use Ollama to run models on your own machine without sending document content to a hosted LLM API.

Some model families to experiment with:



Llama

General-purpose notes, explanations and question generation.



Mistral

Experiment with concise summaries and structured outputs.



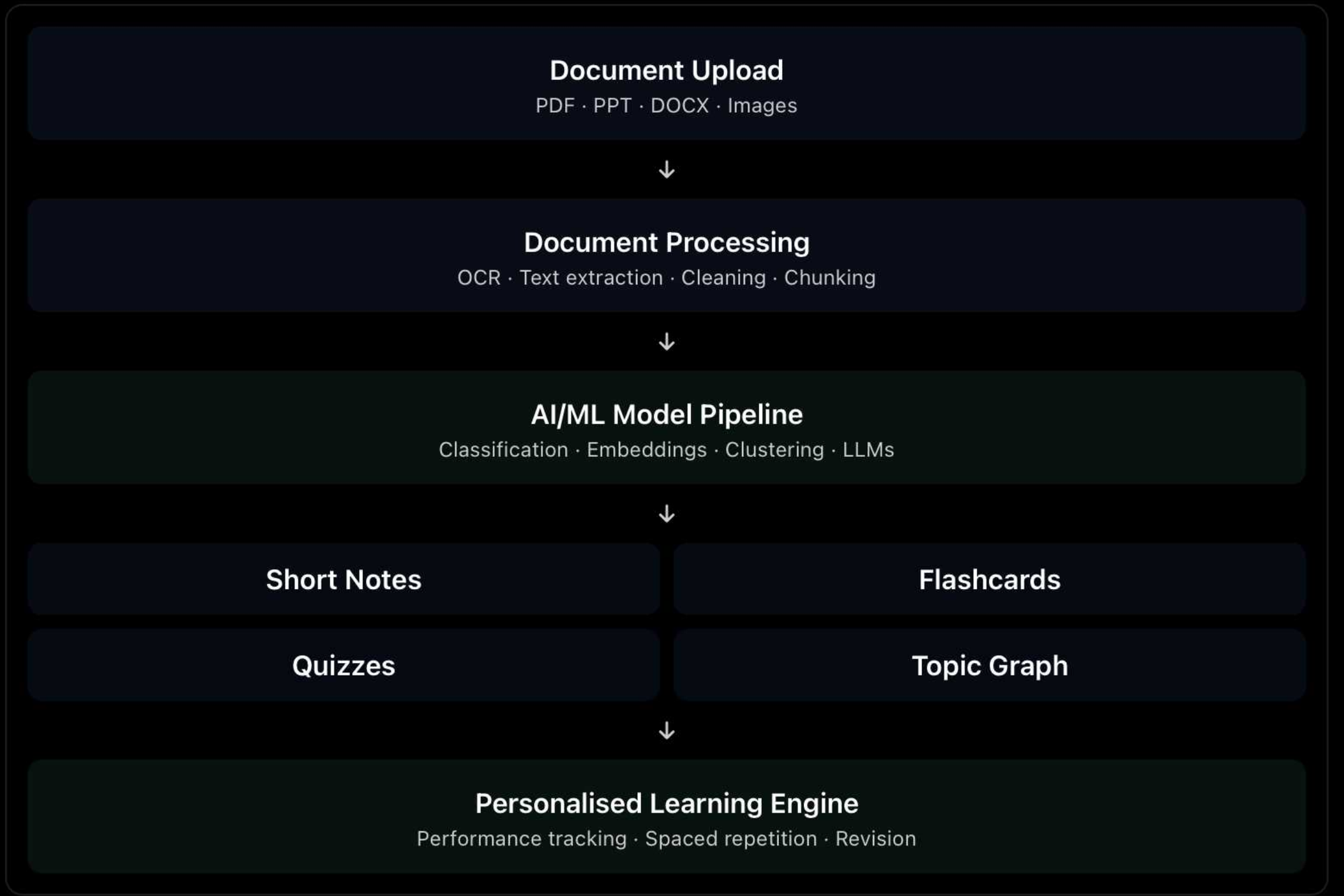
Gemma

Compare question generation, explanations and summarisation.

You can compare models on the same document and measure which produces more accurate notes and useful flashcards. Choose model sizes based on your MacBook's available memory and performance.

Architecture

Here is how I would structure the system:



What will actually demonstrate your AI capabilities?

Don't just say that you integrated five models. Show measurable experiments.

For example:

Experiment	What you measure
Compare three LLMs	Accuracy and quality of generated notes
Compare question-generation models	Relevance and correctness of flashcards
Embedding evaluation	Quality of semantic retrieval
Topic clustering	Silhouette score and cluster quality
Model efficiency	Latency, RAM usage and tokens per second
Hallucination evaluation	Unsupported facts in generated content

Also implement source citations: every generated note or flashcard should ideally link back to the page or passage from which its answer was derived.

Suggested technology stack

Layer	Technology
Backend	Python, FastAPI
Frontend	React, TypeScript
ML	PyTorch, Hugging Face Transformers, scikit-learn
Local LLMs	Ollama
Embeddings	Sentence Transformers
Vector database	FAISS or Chroma
Document processing	PyMuPDF, python-pptx, Tesseract
Database	PostgreSQL
Deployment	Docker

How it compares with your ReproForge AI project

Since you're targeting software engineering internships and want to demonstrate AI skills, here's the distinction:

Dimension	ReproForge AI	NeuroLearn AI
Main domain	AI + Software Engineering	AI + Education
LLM integration	Bug analysis and test generation	Notes and question generation
Traditional ML	Code analysis and fault localisation	Classification and clustering
Model evaluation	Reproduction reliability	Content quality and model comparison
Software engineering	Sandboxing, APIs, execution pipelines	Document pipelines, APIs, databases
User experience	Developer dashboard	Student learning platform

The important distinction: ReproForge demonstrates how you apply AI to software engineering problems. NeuroLearn gives you more visible opportunities to demonstrate traditional ML, NLP, embeddings, local LLMs and model evaluation.

NeuroLearn AI — Advanced AI Flashcard & Smart Notes Platform

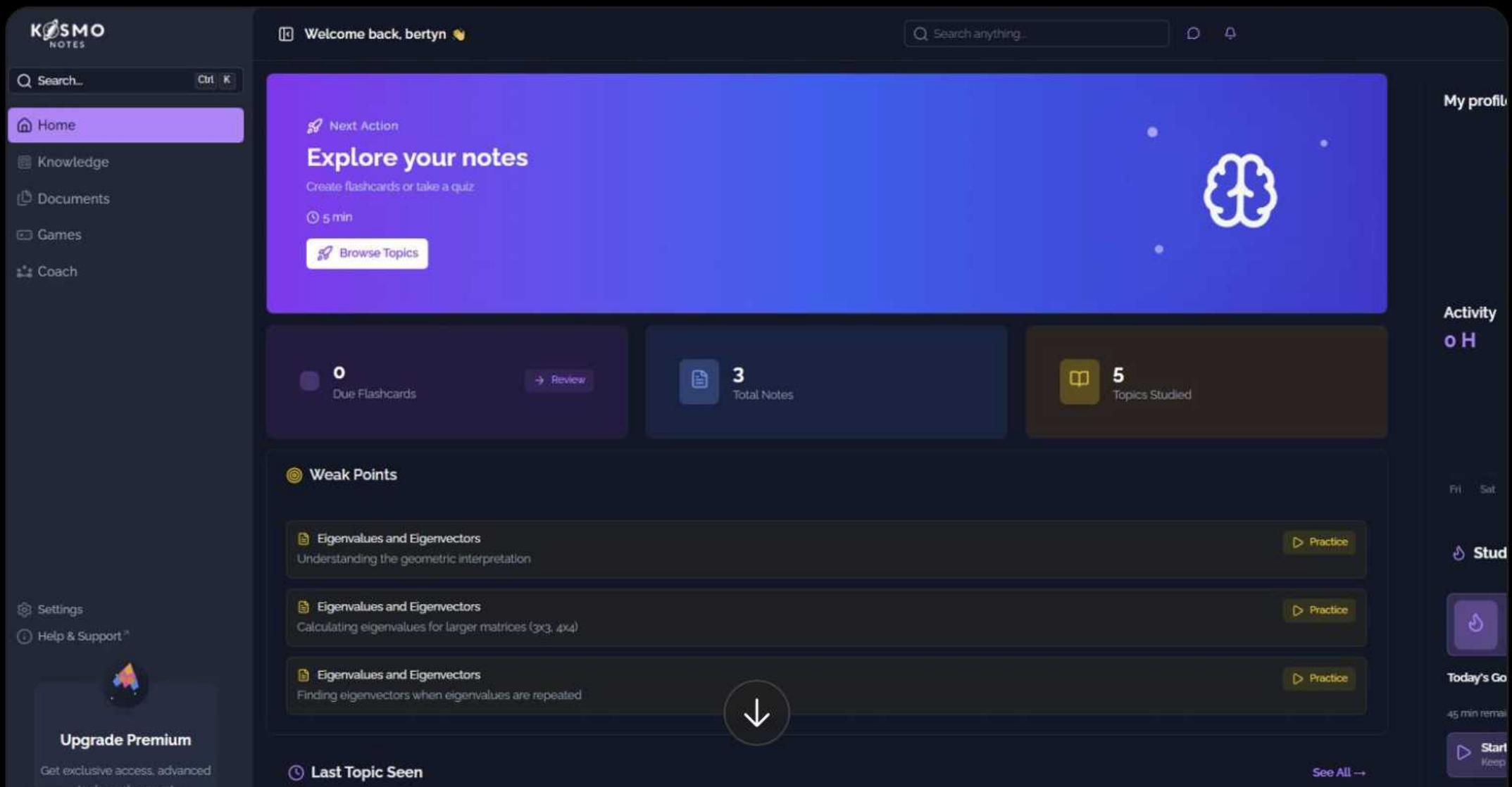
Advanced portfolio project | AI + ML + Software Engineering

I would design this as a serious AI engineering project, not just a website that summarises PDFs.

NeuroLearn AI will be a locally powered intelligent learning platform that converts college study materials into short notes, flashcards, quizzes and personalised revision plans. It will combine traditional machine learning, NLP, local LLMs, semantic search and backend engineering.

The defining feature will be its **multi-model intelligence system**: users can compare different models, see why content was generated, verify answers against their original documents and measure the quality and speed of each model.

1. Project specification



NeuroLearn AI

Your personal AI-powered study workspace

Tagline: Upload. Understand. Remember. Revise.

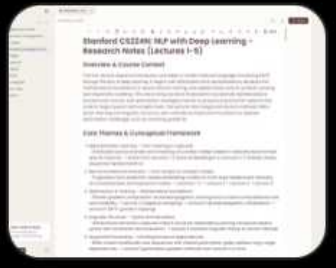
Specification	Design
Target users	College students
Input	PDF, PPTX, DOCX, TXT, scanned notes
Processing	NLP, ML, local LLMs
Output	Notes, flashcards, quizzes, topic maps
Execution	Local-first on MacBook
Backend	Python + FastAPI
Frontend	React + TypeScript
Database	PostgreSQL + vector search
Deployment	Docker

Main user journey

1. A student uploads a lecture PDF or PowerPoint.
2. The system extracts and cleans the text.
3. ML models identify topics, keywords and important concepts.
4. Local LLMs generate short notes and flashcards.
5. An evidence-checking component links generated content to source passages.
6. The student attempts quizzes and flashcards.
7. The learning engine tracks performance and schedules revision.
8. The student can compare different models' output and performance.

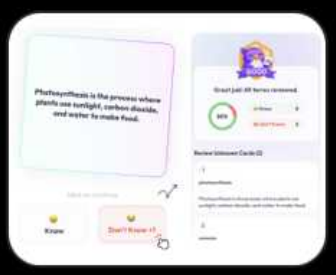
2. Features to build

Core AI features



Smart Notes Generator

- Chapter-wise notes
- Short and detailed summaries
- Important definitions and formulas
- Key points and examples
- Source page references



Intelligent Flashcard Generator

- Question-and-answer cards
- Definition and formula cards
- Conceptual and application-based cards
- Easy, medium and hard difficulty
- Edit, delete and export cards



Adaptive Quiz Engine

- MCQs and short-answer questions
- Topic-specific tests
- Difficulty adjustment
- Explanations for incorrect answers
- Performance reports



Topic Intelligence

- Topic classification
- Similar concept detection
- Related-topic recommendations
- Concept relationship graph



Personalised Revision

- Track correct and incorrect answers
- Topic mastery estimates
- Spaced repetition
- Daily revision queue
- Progress analytics

Features that make it stand out

These are the features I would prioritise after the core application works:

- **Multi-model comparison:** Generate the same notes with two or three models and compare results.
- **Evidence-grounded generation:** Every generated fact should point to the source page and passage.
- **Hallucination detection:** Flag answers that cannot be supported by the uploaded material.
- **Model evaluation dashboard:** Compare generation quality, response time, memory usage and output length.
- **Local-first privacy:** Process documents locally by default, without requiring an external LLM API.

3. Model selection: what runs locally?

Your MacBook Air M4 is suitable for developing a local-first version. However, the amount of unified memory matters more than the processor name when choosing model sizes. Start with small quantised models and measure actual memory use before trying larger ones.

Ollama supports Apple M-series Macs, including CPU and GPU acceleration. Model files can take tens of gigabytes or more, so manage storage carefully. GitHub

A. Local LLMs

I recommend comparing three model families rather than downloading many models immediately.

Model family	Role in NeuroLearn	What to test
Qwen	Notes, structured flashcards, quizzes	Instruction following and structured output
Gemma	Summarisation and explanations	Clarity and factual consistency
Llama	General-purpose generation	Quality and response speed

These are candidates for experiments, not claims that one is automatically better. Record the exact model tag, quantisation, context size and Ollama version in your results.

Starting model sizes: try a 3B–4B quantised model first. Only move to larger models if your Mac's memory and latency measurements allow it.

B. Traditional ML and NLP models

This is where you demonstrate that the project uses more than LLM prompting.

Task	Suggested model / method	Why it is useful
Topic classification	DistilBERT	Categorise lecture content
Keyword extraction	KeyBERT	Extract important concepts
Semantic embeddings	all-MiniLM-L6-v2	Find similar concepts and passages
Reranking	cross-encoder/ms-marco-MiniLM-L6-v2	Rank relevant passages
Topic clustering	K-Means / HDBSCAN	Group related concepts
Difficulty classification	Fine-tuned DistilBERT	Predict question difficulty
Spaced repetition	SM-2 algorithm	Schedule revision

Sentence Transformers provides embedding and cross-encoder models. A cross-encoder can score a query-passage pair for relevance, making it useful for selecting supporting passages for a generated answer. GitHub +1

C. Which models should you actually implement first?

Keep the first version manageable:

Initial model lineup

Build these first

- One small Qwen instruct model through Ollama.
- One Gemma or Llama instruct model through Ollama.
- `sentence-transformers/all-MiniLM-L6-v2` for embeddings.
- `cross-encoder/ms-marco-MiniLM-L6-v2` for passage reranking.
- KeyBERT for keyword extraction.
- K-Means for topic grouping.

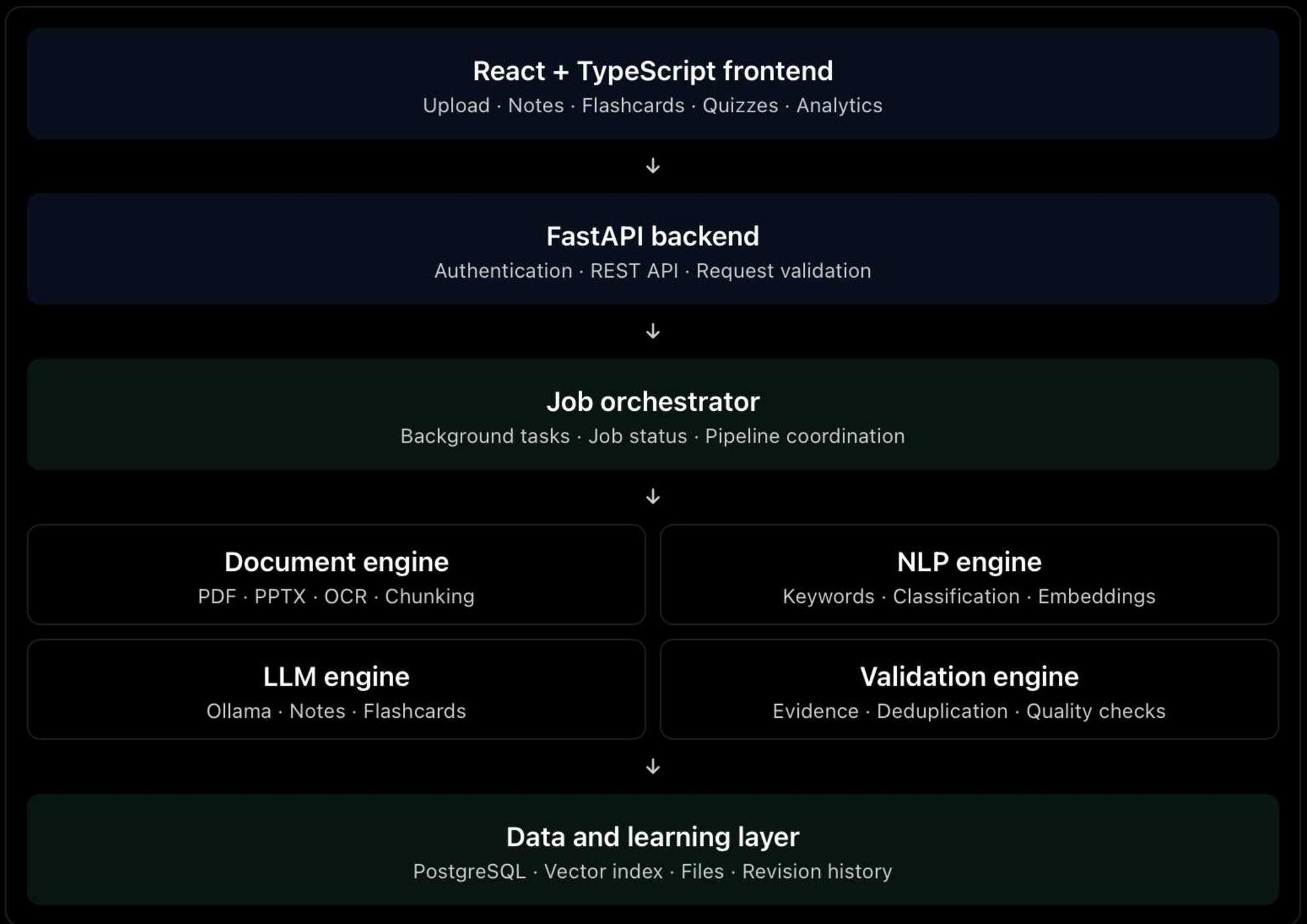
Add later

- DistilBERT fine-tuning for topic and difficulty classification.
- A dedicated question-generation model.
- HDBSCAN for clustering without specifying the number of clusters.

Do not fine-tune a large language model at the beginning. First build a good evaluation dataset and establish how well the pretrained models perform.

4. System architecture

The architecture should separate document processing, model inference, content validation and learning analytics. That separation will make it easier to test individual components and replace models without rewriting the application.



How a document moves through the system

Stage	Operation	Output
1	Upload document	Original file
2	Extract text and page information	Structured text
3	Clean and split text	Source chunks
4	Extract keywords and classify topics	Topic metadata
5	Generate embeddings	Vector representations
6	Generate notes and cards	Structured AI output
7	Retrieve source evidence	Supporting passages
8	Validate and deduplicate	Approved or flagged content
9	Save to database	Notes, cards, citations
10	Start revision tracking	Learning history

An important engineering decision

Do not make the frontend wait for a long document-processing request.

Use a background job system:

- FastAPI receives the upload.
- It creates a job with a unique ID.
- A worker processes the document.
- The frontend polls or subscribes to job progress.
- The completed result is saved and displayed.

For the first version, FastAPI background tasks can be sufficient. For a more robust system, use Celery with Redis.

5. The AI pipeline in detail

Document intelligence

Use PyMuPDF for PDF extraction, `python-pptx` for slides, `python-docx` for Word documents and Tesseract for scanned pages.

Each extracted chunk should preserve:

- Document ID
- Page or slide number
- Heading
- Text
- Character offsets
- Extraction method
- OCR confidence, when applicable

For example:

JSON

```json
{
  "document_id": "doc_001",
  "page_number": 12,
  "heading": "Neural Networks",
  "text": "A neural network consists of...",
  "chunk_id": "chunk_012"
}
```

This metadata is essential for source citations.

Topic and keyword intelligence

Run KeyBERT to identify important phrases. Use an embedding model to group semantically similar passages. Then use a classifier or clustering algorithm to identify topic groups.

For example, a machine-learning lecture might produce:

```
Machine Learning
├── Supervised Learning
│   ├── Classification
│   └── Regression
├── Unsupervised Learning
│   ├── Clustering
│   └── Dimensionality Reduction
└── Model Evaluation
    ├── Precision
    ├── Recall
    └── F1 Score
```

Initially, generate this structure from clustering and extracted headings. Don't assume that an unsupervised cluster automatically represents a correct academic topic.

Notes and flashcard generation

Use structured prompts and validate the LLM output against a schema.

Example flashcard format:

JSON

```json
{
  "question": "What is supervised learning?",
  "answer": "A machine learning approach that learns from labelled training data.",
  "topic": "Machine Learning",
  "difficulty": "easy",
  "source_pages": [12],
  "evidence_chunk_ids": ["chunk_012"]
}
```

The system should reject malformed output, missing evidence references and cards with duplicate questions.

Evidence verification

This should be one of your signature features.

For every generated card:

1. Extract its answer and important factual claims.
2. Search the document's vector index for supporting passages.
3. Rerank the retrieved passages.
4. Check whether the evidence supports the answer.
5. Mark the card as supported, partially supported or unverified.
6. Let the user inspect the original page.

A similarity score alone is not proof of factual correctness. Use it for retrieval, then use a separate validation step and retain the source passage for human inspection.

6. Software engineering design

This project should look like a maintainable software product, not a collection of notebooks.

Backend modules

Module	Responsibility
`document_service`	Upload, validate and manage documents
`extraction_service`	PDF, PPTX, DOCX and OCR extraction
`chunking_service`	Split text and preserve page references
`nlp_service`	Keywords, classification and clustering
`embedding_service`	Generate and store vectors
`llm_service`	Communicate with local models
`generation_service`	Create notes, flashcards and quizzes
`validation_service`	Check evidence, schema and duplicates
`learning_service`	Revision schedules and mastery tracking
`evaluation_service`	Model benchmarks and experiment results

Model adapter design

Do not hardcode Ollama calls inside your notes generator. Create a common interface so that you can switch between models easily.

For example:

```python
class LLMProvider:
    def generate(self, prompt: str, schema: dict) -> dict:
        raise NotImplementedError
```

Then implement:

```python
class OllamaProvider(LLMProvider):
    def generate(self, prompt: str, schema: dict) -> dict:
        # Call local Ollama model
        pass
```

Your generation service should work with the interface rather than a specific model. This demonstrates abstraction, modularity and testability.

Suggested API endpoints

Method	Endpoint	Purpose
POST	`/api/documents/upload`	Upload study material
GET	`/api/documents`	List documents
GET	`/api/jobs/{job_id}`	Check processing status
POST	`/api/notes/generate`	Generate notes
GET	`/api/notes/{document_id}`	Retrieve notes
POST	`/api/flashcards/generate`	Generate flashcards
GET	`/api/flashcards/{document_id}`	Retrieve cards
POST	`/api/quizzes/generate`	Generate quiz
POST	`/api/revision/submit`	Submit flashcard response
GET	`/api/revision/today`	Get today's revision queue
GET	`/api/evaluation/compare`	Compare model results

Database design

Start with PostgreSQL. Use `pgvector` if you want vector storage within the same database; alternatively, use FAISS for a local prototype.

Your main entities should be:

Table	Important fields
users	id, name, email
documents	id, user_id, filename, status
chunks	id, document_id, page, text
topics	id, document_id, topic_name
embeddings	chunk_id, vector
notes	id, document_id, content
flashcards	id, topic_id, question, answer, difficulty
card_evidence	card_id, chunk_id, support_status
quiz_attempts	id, user_id, score, created_at
revision_history	card_id, user_id, rating, next_review
model_runs	model_name, task, latency, memory, output

7. Research and experimentation

Even though this is an advanced portfolio project rather than a formal research paper, it should contain real experiments.

Your project should answer:

How do different local AI models perform when generating accurate, useful and evidence-supported study materials from college lecture documents?

Experiment A — LLM comparison

Use the same set of documents and prompts with three local models.

Measure:

- Accuracy of generated notes
- Coverage of important concepts
- Factual consistency
- Relevance of flashcards
- Quality of explanations
- Generation time
- Peak memory usage

Keep model settings and prompts consistent where possible. Have human reviewers assess a sample of outputs using a defined rubric.

Experiment B — Flashcard quality

Compare different generation approaches:

Approach	Description
A	Direct LLM flashcard generation
B	Keyword extraction followed by LLM generation
C	Topic extraction + LLM generation + evidence verification

Measure:

- Question relevance
- Answer correctness
- Duplicate card rate
- Source support rate
- Coverage of the original document

This tests whether the additional ML stages provide measurable value.

Experiment C — Embedding and retrieval comparison

Compare two or more embedding models on a set of questions with manually identified supporting passages.

Measure:

- Recall@5
- Mean Reciprocal Rank (MRR)
- Retrieval latency
- Evidence relevance

Experiment D — Hallucination and evidence support

Create a test set containing:

- Answers directly stated in the document
- Answers requiring information from multiple sections
- Questions not answered in the document
- Questions based on misleading or ambiguous wording

Measure whether the platform correctly identifies unsupported answers rather than confidently inventing an answer.

Experiment E — Local performance

Measure each model on your MacBook:

- Model loading time
- First-token latency
- Total generation time
- Tokens per second
- Peak memory usage
- Model file size
- CPU/GPU utilisation, where measurable

Run repeated trials, record the environment and report median and variation. Do not assume that a larger model will necessarily give better results on your specific tasks.

Build a small benchmark dataset

Start with 10–15 lecture documents across different college subjects, then expand toward 30–50 if time allows.

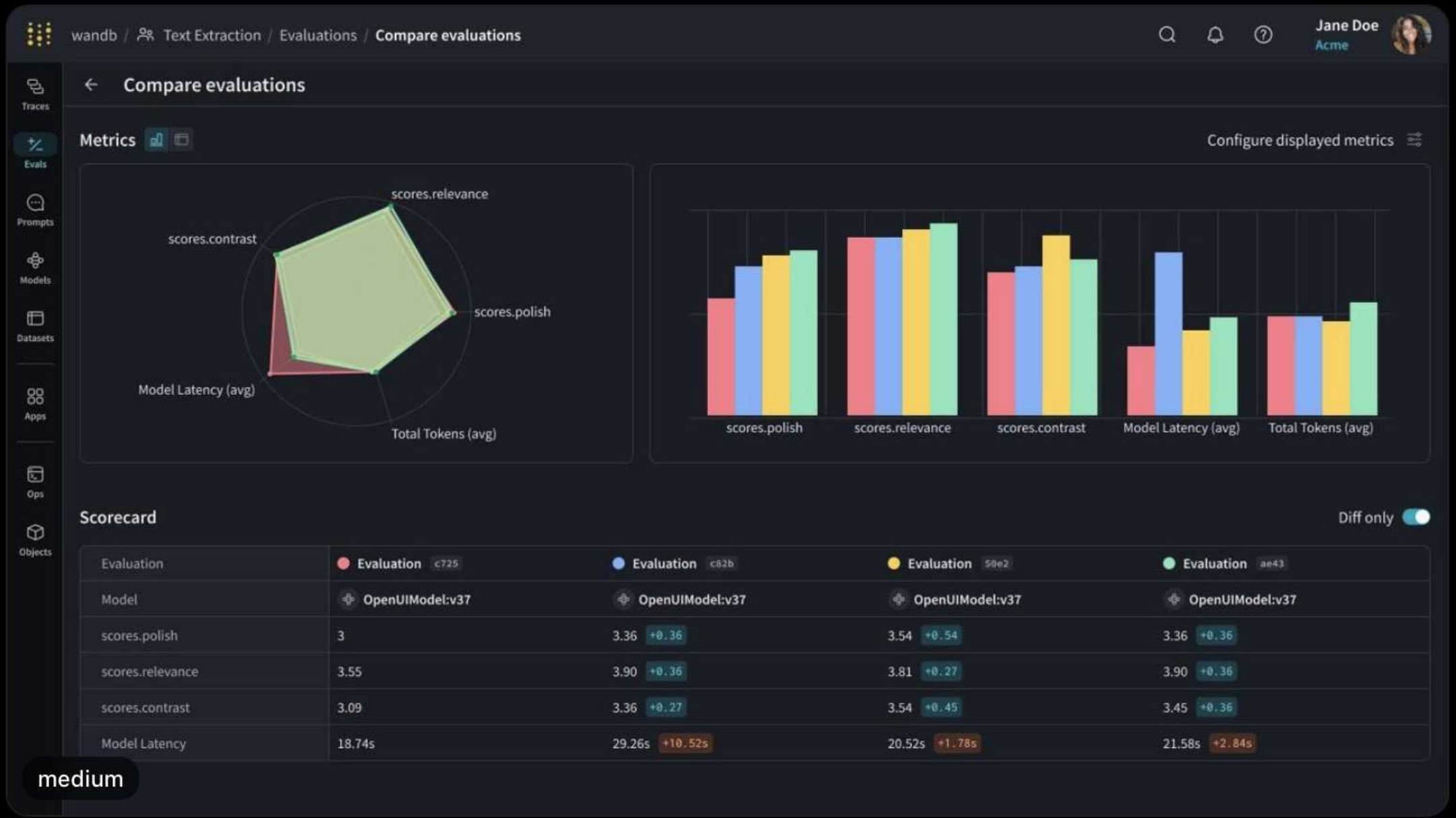
For each document, manually prepare:

- Important topics
- Reference notes
- 10–20 expected questions
- Correct answers
- Supporting page numbers
- Questions that cannot be answered from the source

This benchmark becomes one of the most valuable parts of your portfolio because it gives you evidence to support your model-selection decisions.

8. Evaluation dashboard

Build a separate page where users can compare model outputs.



medium

Suggested dashboard sections:

Suggested dashboard sections:

Metric	Display
Notes quality	Human evaluation score
Flashcard quality	Relevance and correctness
Evidence support	Percentage of supported claims
Retrieval quality	Recall@5 and MRR
Performance	Generation time and tokens/sec
Resource usage	Peak RAM and model size

Use the dashboard to show actual results from your experiments. Do not put made-up model accuracy percentages in the interface.

9. Personalised learning engine

This feature takes the application beyond content generation.

Start with the SM-2 spaced-repetition algorithm. Each flashcard receives a review interval based on the user's response.

Example user ratings:

- Again — I couldn't remember.
- Hard — I remembered with difficulty.
- Good — I remembered correctly.
- Easy — I remembered immediately.

Store each response, calculate the next review date and create a daily revision queue.

Later, add a topic mastery estimate using the student's performance history. Keep mastery separate from simple quiz accuracy: a student who answers one question correctly has not necessarily mastered an entire topic.

10. Step-by-step implementation roadmap

I recommend a **10-week development plan**, with each week producing something demonstrable.

1. **Week 1 — Project foundation**
   - Create GitHub repository.
   - Set up Python virtual environment.
   - Create FastAPI backend.
   - Create React + TypeScript frontend.
   - Install Ollama and test one small local model.
   - Build a basic upload page.

   Deliverable: Working application skeleton

2. **Week 2 — Document processing**
   - PDF extraction using PyMuPDF.
   - PPTX extraction using python-pptx.
   - DOCX extraction.
   - OCR for scanned documents.
   - Text cleaning and chunking.
   - Preserve page and heading information.

   Deliverable: Document processing pipeline

3 Week 3 — NLP and traditional ML

- Keyword extraction using KeyBERT.
- Generate embeddings.
- Implement topic clustering.
- Add topic classification.
- Display extracted topics in the frontend.

Deliverable: Topic intelligence engine

4 Week 4 — Local LLM integration

- Connect Ollama with FastAPI.
- Integrate two local models.
- Generate structured short notes.
- Add model selection.
- Record model execution time.

Deliverable: Multi-model notes generator

5 Week 5 — Flashcard generation

- Generate question-answer pairs.
- Generate different question types.
- Add difficulty levels.
- Validate structured output.
- Detect duplicate questions.

Deliverable: Flashcard generation engine

6 **Week 6 — Evidence and semantic retrieval**

- Store document embeddings.
- Implement semantic search.
- Add cross-encoder reranking.
- Link generated cards to source pages.
- Flag unsupported answers.

Deliverable: Evidence-grounded AI generation

7 **Week 7 — Quiz and learning engine**

- Generate MCQs.
- Record quiz results.
- Implement spaced repetition.
- Build a revision queue.
- Show topic-wise progress.

Deliverable: Personalised learning engine

8 **Week 8 — Model evaluation**

- Prepare benchmark documents.
- Compare local LLMs.
- Evaluate flashcard quality.
- Evaluate retrieval.
- Measure latency and memory usage.
- Create comparison dashboard.

Deliverable: AI evaluation system

9 **Week 9 — Software engineering quality**

- Add authentication and user-specific document access.
- Add PostgreSQL persistence.
- Implement background jobs.
- Write unit and integration tests.
- Add Docker and Docker Compose.
- Add error handling and logging.

Deliverable: Reliable application

10 **Week 10 — Final product**

- Polish frontend.
- Add loading and error states.
- Add sample documents and demo account.
- Write architecture documentation.
- Publish experiment results.
- Record a short demonstration video.
- Prepare GitHub README.

Deliverable: Portfolio-ready project

11. GitHub repository structure

Organise the project from the beginning so it is easy for another developer or recruiter to understand.

```
neurolearn-ai/
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── core/
│   │   ├── database/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── ml/
│   │   │   ├── embeddings/
│   │   │   ├── classification/
│   │   │   ├── clustering/
│   │   │   └── keywords/
│   │   ├── llm/
│   │   │   ├── providers/
│   │   │   ├── prompts/
│   │   │   └── generation/
│   │   ├── evaluation/
│   │   └── main.py
│   │
│   └── tests/
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── types/
│
├── benchmark/
```

```
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── types/
│
├── benchmark/
│   ├── documents/
│   ├── reference_questions/
│   ├── reference_answers/
│   └── evaluation_results/
│
├── experiments/
│   ├── llm_comparison/
│   ├── retrieval/
│   ├── flashcard_quality/
│   └── performance/
│
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── model_selection.md
│   └── experiments.md
│
├── docker-compose.yml
├── requirements.txt
├── .env.example
└── README.md
```

Never upload API keys, private documents, personal learning records or downloaded model binaries to GitHub. Keep model downloads separate from your source repository.

12. Security and privacy

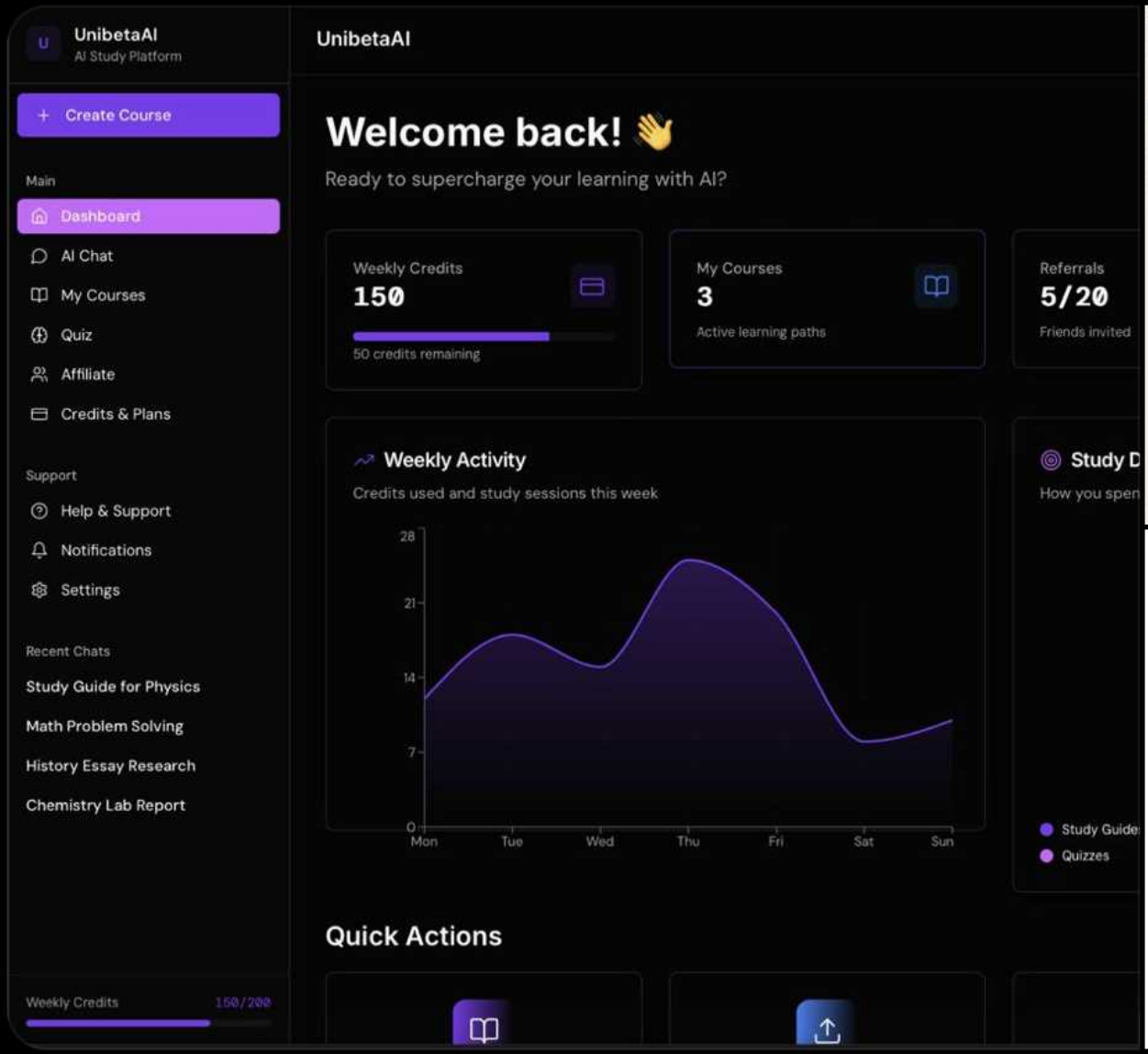
Since you want local execution, privacy can be a meaningful product feature.

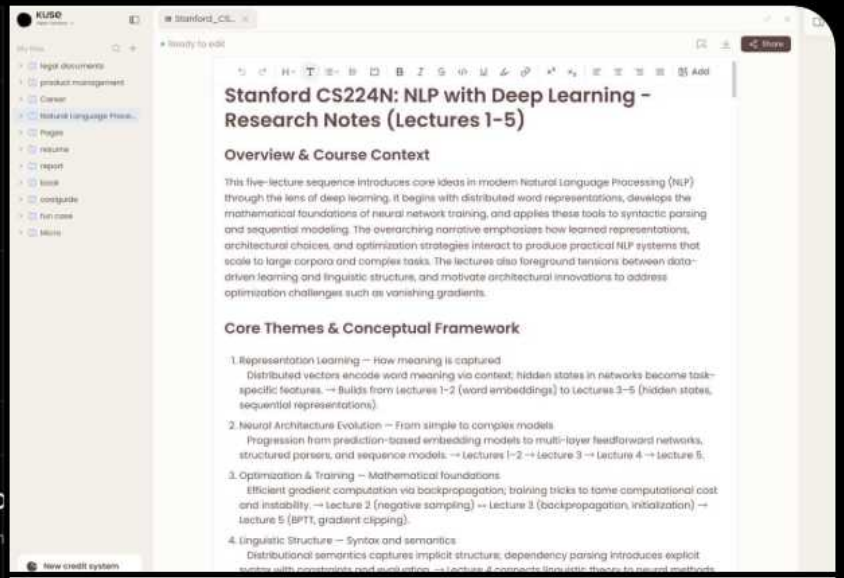
Implement these safeguards:

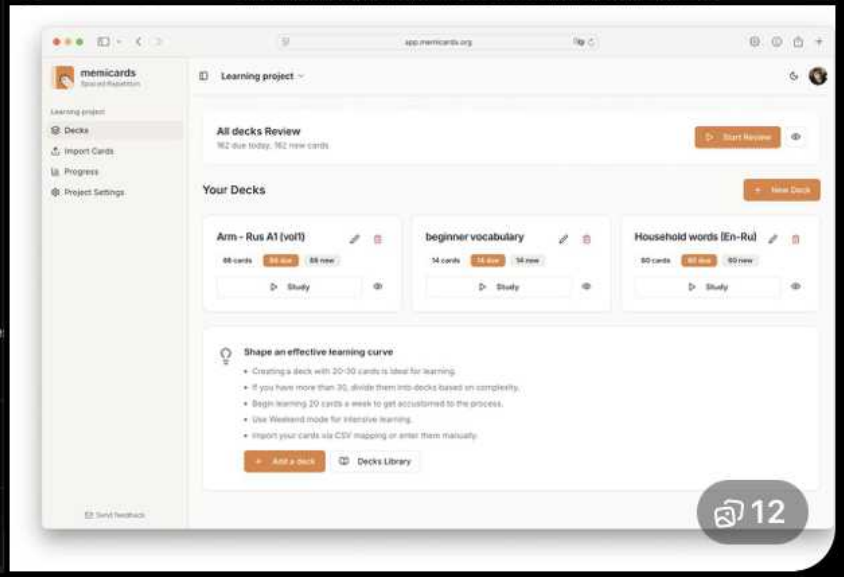
- Process uploaded documents locally by default.
- Restrict document access to the owning user.
- Validate file types and file sizes.
- Treat extracted document text as untrusted input; it may contain prompt injection attempts.
- Never let document content override system instructions or trigger arbitrary tool execution.
- Avoid sending private content to external services.
- Store credentials in environment variables.
- Use access-controlled file storage.
- Delete temporary extraction files after processing when no longer needed.

A local LLM does not automatically make the whole application private. Your frontend, database, logs, file storage and any external dependencies must also be considered.

13. What should the final product look like?







I would build five main pages:

Page	Main purpose
Dashboard	Recent documents, study progress and revision queue
My Materials	Upload and organise PDFs and lecture slides
Smart Notes	Read, edit and export generated notes
Flashcards & Quizzes	Study, test and review concepts
Model Lab	Compare models and inspect evaluation results

The Model Lab is particularly important: it makes your AI work visible rather than hiding all the ML behind a single Generate button.

14. What you can put on your resume

Once the project is genuinely implemented and evaluated, this is a suitable starting description.

Writing

NeuroLearn AI – Multi-Model Intelligent Study Assistant | Python, FastAPI, React, PyTorch, Hugging Face, Ollama, Sentence Transformers, PostgreSQL

- Developed a local-first AI learning platform that transforms PDFs, lecture slides and study materials into structured notes, flashcards and adaptive quizzes using multiple ML and local LLM models.
- Implemented an NLP pipeline incorporating keyword extraction, semantic embeddings, topic classification and clustering for intelligent document understanding.
- Engineered evidence-grounded content generation with semantic retrieval, passage reranking and source-page references to help verify generated answers.
- Built a model evaluation framework to comp LMs on content quality, factual support, generation latency and local resource consumption.

- Built a model evaluation framework to compare LLMs on content quality, factual support, generation latency and local resource consumption.
- Designed a personalised revision engine using spaced repetition, quiz analytics and topic-level performance tracking.

Only retain bullets that you have actually implemented. Add real benchmark results when available.

15. What will make this project stand out?

The difference between a basic application and an advanced AI engineering project is what happens behind the interface.

Basic implementation	Advanced implementation
One LLM generates notes	Multiple models with measurable comparisons
Flashcards are generated	Cards are evaluated for quality and coverage
Notes show plain text	Notes contain source-page references
Embeddings are stored	Retrieval is benchmarked and reranked
Quiz score is displayed	Learning history drives revision scheduling
Models are selected manually	Model Lab records latency and resource use
Application works locally	Tested, containerised and documented architecture